University of Buea

Faculty of Engineering and Technology

INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Group Presentation

App development notions

TASK 1

Overview of Presentation

1. Types of Mobile Apps
2. Programming Languages
3. Frameworks
4. Architectures & Design Patterns

Cost Estimation

5. Requirements Engineering
6. Conclusion

Types of Mobile Apps

1

Key Concept

Choose based on performance needs, budget, and target platforms. Native offers best UX, Hybrid saves development time and cost.

2

Native Apps

Built for a single OS (iOS or Android). Best performance and full hardware access. Examples: Swift for iOS, Kotlin for Android.

3

Progressive Web Apps

Web-based apps installable via browser. Work offline with service workers. Limited hardware access but no app store required.

4

Hybrid Apps

Web apps wrapped in native shell. One codebase for multiple platforms. Built with Ionic, Cordova, or Capacitor frameworks.

Types of Apps Comparison

1

Native Apps

• Performance: Excellent - optimized for specific OS
• Development Cost: Higher - separate codebases needed
• Hardware Access: Full - camera, GPS, sensors, etc.
• App Store: Yes - available on iOS/Android stores

2

PWA & Hybrid Apps

• PWA Performance: Good - browser-based limitations
• Hybrid Performance: Moderate - web wrapper overhead
• Development Cost: Lower - single codebase
• Hardware Access: Limited (PWA) / Moderate (Hybrid)
• App Store: PWA No / Hybrid Yes

Programming Languages

Native: Java (legacy Android), Kotlin (modern Android), Swift (iOS) - platform-specific, optimal performance

---

Cross-Platform: Dart (Flutter), JavaScript/TypeScript (React Native) - single codebase approach

---

Hybrid: HTML, CSS, JavaScript/TypeScript with Ionic, Capacitor, or Cordova frameworks

---

Key Takeaway: Kotlin and Swift lead native development; Dart and JavaScript lead cross-platform

Frameworks Overview

1

Flutter

High performance framework using Dart. Offers beautiful, customizable UI with a single codebase for iOS and Android.

2

React Native

Uses JavaScript/TypeScript. Easy to learn with strong community support and extensive third-party libraries.

3

Xamarin

Microsoft's C# framework. Ideal for enterprise applications with native performance and .NET integration.

4

Ionic

Web-based framework using HTML, CSS, and JavaScript. Enables fast development with lower performance overhead.

Framework Comparison

1

Flutter vs React Native

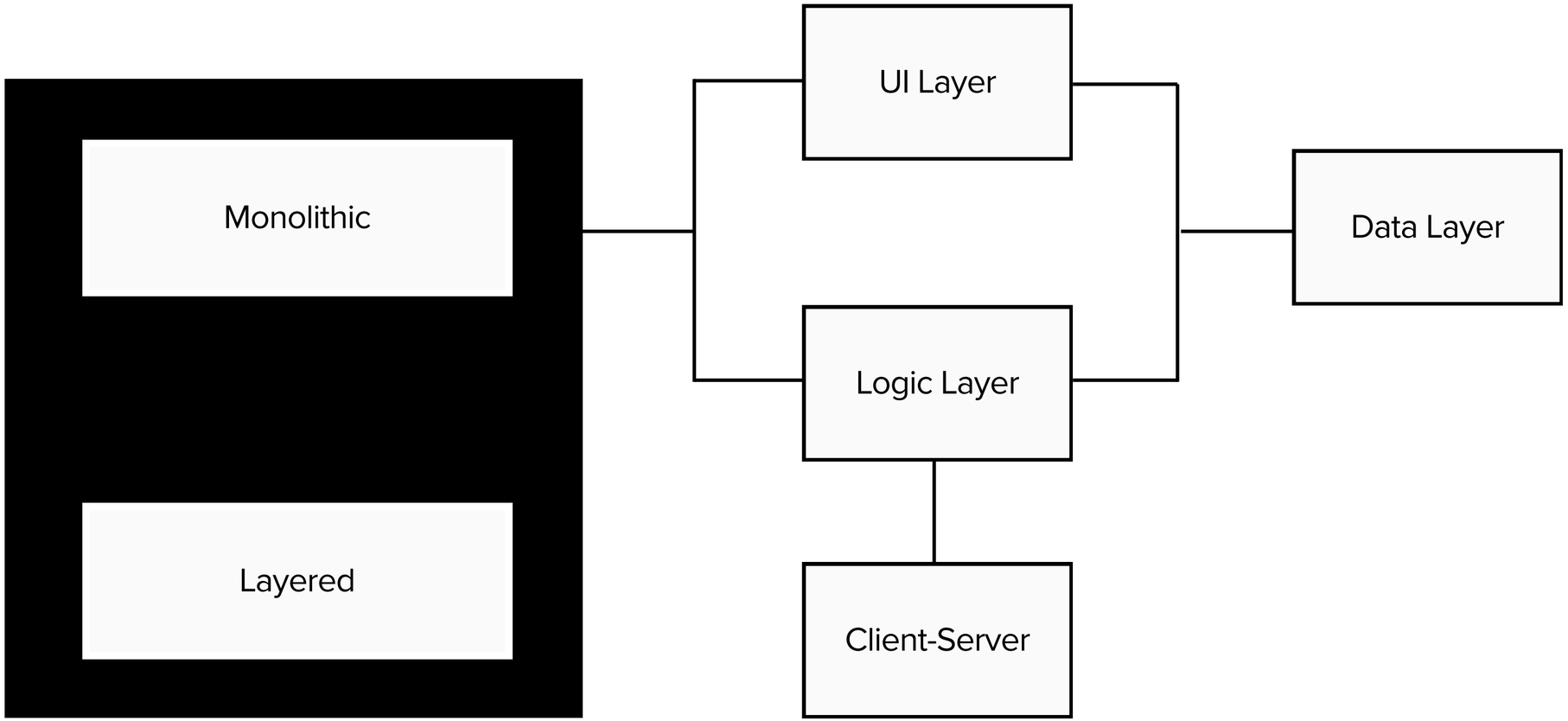
Flutter: Uses Dart language, offers high performance with native compilation, moderate cost and development time, excellent UI/UX with customizable widgets.
React Native: Uses JavaScript, good performance via native bridges, lower cost with faster development, strong UI/UX with native components.

2

Xamarin vs Ionic

Xamarin: Uses C# language, near-native performance, higher cost suited for enterprise, native UI/UX capabilities.
Ionic: Uses HTML/CSS/JavaScript, lower performance (web-based), lowest cost with rapid development, web-based UI with native styling.

Mobile App Architectures



Design Patterns

MVC (Model-View-Controller): Simple and widely used pattern separating data, UI, and control logic.

MVP (Model-View-Presenter): Better separation of concerns, easier unit testing than MVC.

MVVM (Model-View-ViewModel): Modern pattern supporting data binding — most popular for contemporary apps.

VIPER & Others: VIPER for large iOS apps; Repository for data management; Singleton for global access.

Choosing the Right Architecture

Small Apps

MVC or Layered architecture for simple, straightforward applications

Large Apps

MVVM + Repository or Microservices for scalable enterprise solutions

Requirements Engineering

1

Collection

Gather requirements through interviews, surveys, observation, focus groups, and brainstorming sessions with stakeholders.

2

Analysis

Classify requirements as functional or non-functional. Prioritize based on importance and model using use cases and user stories.

3

Validation

Verify requirements through prototyping, user feedback collection, and systematic reviews and testing processes.

4

Goal

Ensure user-centered, quality applications by aligning development with validated user needs and expectations.

Cost Estimation Formula

Understanding Development Cost Calculation

The Formula

Cost = (Total Development Hours × Hourly Rate) + External Costs

This fundamental equation captures the core components of mobile app development pricing. Development hours depend on feature complexity and team size, while hourly rates vary significantly by region and expertise level.

Key Influencing Factors

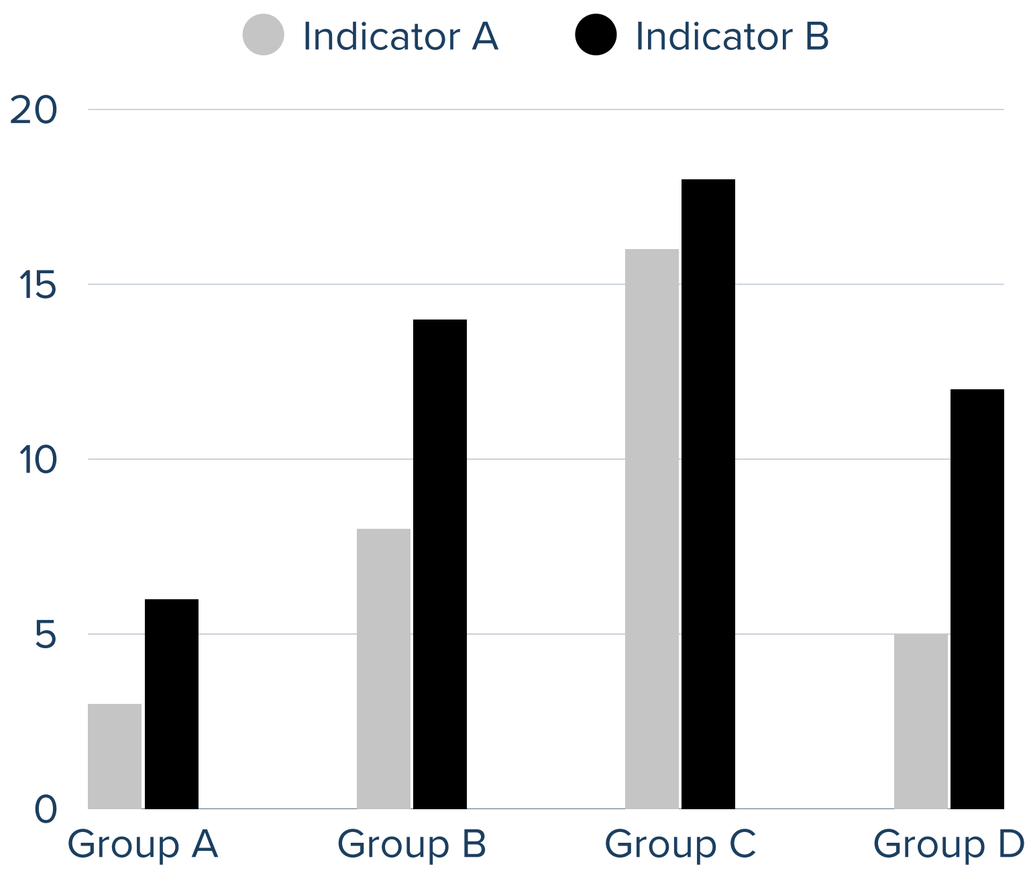
• App Complexity: Simple apps (100-300 hrs), Medium (300-700 hrs), Complex (700+ hrs)

• Platform Choice: Single platform vs cross-platform development

• Developer Location Rates:

- US/Canada: $100-250/hr
- Western Europe: $80-180/hr
- Eastern Europe: $40-100/hr
- Asia: $20-80/hr

Cost Breakdown by Phase



Budget distribution across development phases:

- Research (10%): Market analysis, competitor study
- Design (15-20%): UI/UX, wireframes, prototypes
- Development (50-60%): Core coding, integration
- Testing (15%): QA, bug fixes, user testing
- Deployment (5%): App store submission, launch

Hidden and Ongoing Costs

Maintenance

15-25% of original cost per year. Covers bug fixes, OS updates, security patches, and feature enhancements.

Marketing Costs

Can equal or exceed development cost. Includes user acquisition, app store optimization, and promotional campaigns.

Beyond initial development, mobile apps require continuous investment. Understanding these hidden costs is crucial for accurate budgeting and long-term project sustainability.

Conclusion

Key Takeaways

Cross-platform frameworks like Flutter and React Native offer the optimal balance between development cost and application performance. MVVM architecture stands out as the ideal choice for modern mobile applications, providing clean separation of concerns and excellent data binding support.

Path to Success

Proper requirements analysis ensures user-centered design and reduces costly revisions. Accurate cost estimation, including hidden and ongoing expenses, is essential for successful project delivery. Together, these practices form the foundation of effective mobile app development.

THANK YOU